

Module 9

Harness Continuous Delivery

Deployment Strategies, Verification & Rollback



CI answers: Can I build and test this version? **CD answers:** How do I safely release this version to users?

Learning Objectives

By the end of this module, students will be able to:



Explain the purpose of Continuous Delivery



Configure a Kubernetes deployment stage



Understand deployment verification



Explain automatic and manual rollback



Understand the Harness CD deployment model



Compare Rolling, Blue Green & Canary strategies



Configure approvals and failure strategies



Select the right strategy for production

From CI to CD



CI Pipeline includes

- ◆ Compile
- Unit Test
- SAST
- Image Build
- Image Scan

Harness CD includes

- Select Version
- Deploy
- Verify
- Approve
- Promote

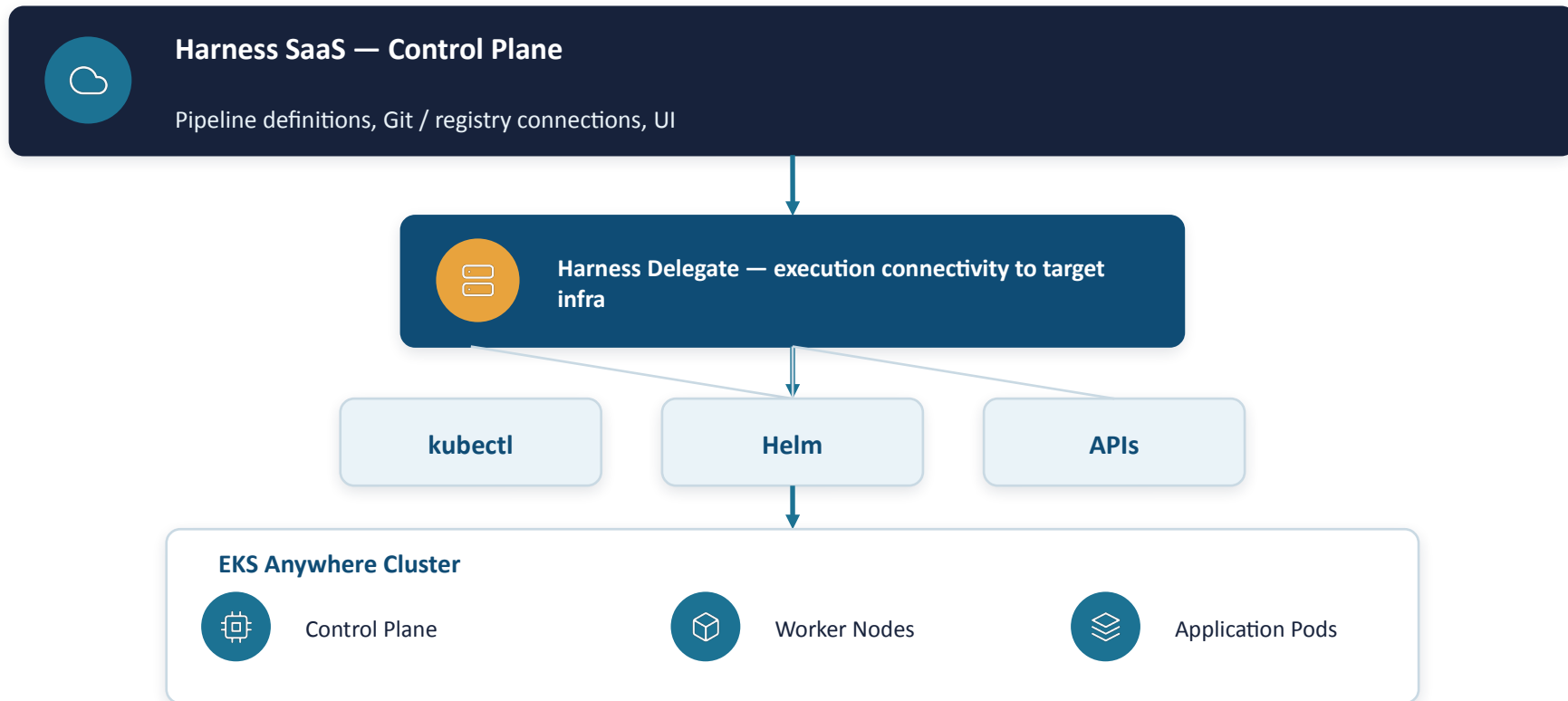
CI

"Is this build technically ready?"

CD

"Can this version be safely released?"

Harness CD Architecture



What Is a Harness CD Stage?



Service

Application, artifact, K8s manifests,
config & variables



Environment

Dev, QA, Stage, Production



Infrastructure

Kubernetes cluster, namespace,
deployment target



Execution Strategy

Rolling, Blue Green, Canary



Verification

Health & observability checks post-
deploy



Approval

Manual or automated gate before
promotion



Rollback

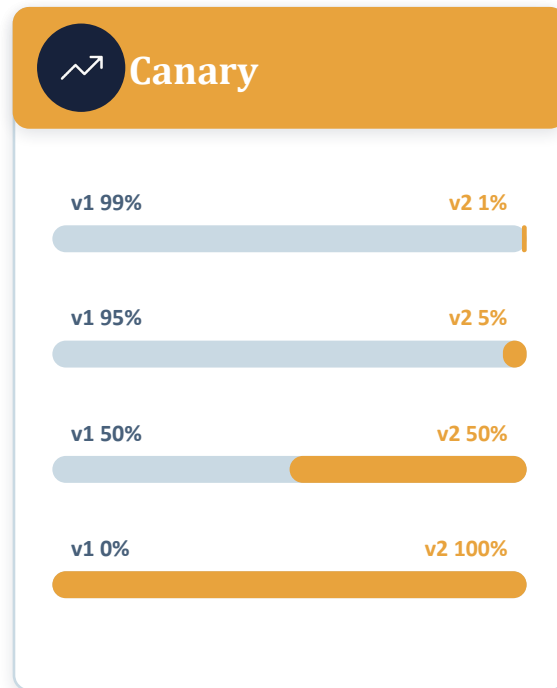
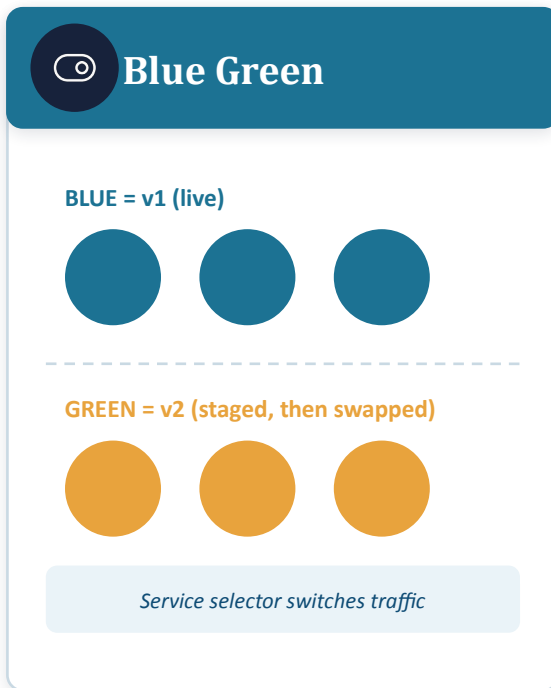
Automatic or manual failure
recovery

*Service + Environment define
WHAT and WHERE you deploy.*

SAME APPLICATION, DIFFERENT RISK

Why Deployment Strategy Matters

v1.0 (6 replicas) is running. We want to release **v2.0**. How should traffic move?



Rolling Deployment

Gradually replaces old application instances with the new version.



Advantages

- ✦ Simple
- ▮ Resource efficient
- ▮ Native Kubernetes model
- ▮ Good default strategy
- ▮ No duplicate environments needed



Risks

Two versions coexist during rollout (v1 + v2).

The application must tolerate temporary mixed-version execution.

Harness Kubernetes rolling strategy incrementally updates pods and waits for the workload to reach steady state.

Kubernetes Rolling Update Mechanics

Deployment maintains two ReplicaSets

ReplicaSet v1

v1

v1

v1

ReplicaSet v2

v2

replicas: 4 · strategy: RollingUpdate

v1 = 4 → 3 → 2 → 1 → 0

v2 = 0 → 1 → 2 → 3 → 4

Key Parameters

maxSurge: 1

maxUnavailable: 0

Never take an available pod away before its replacement is ready.

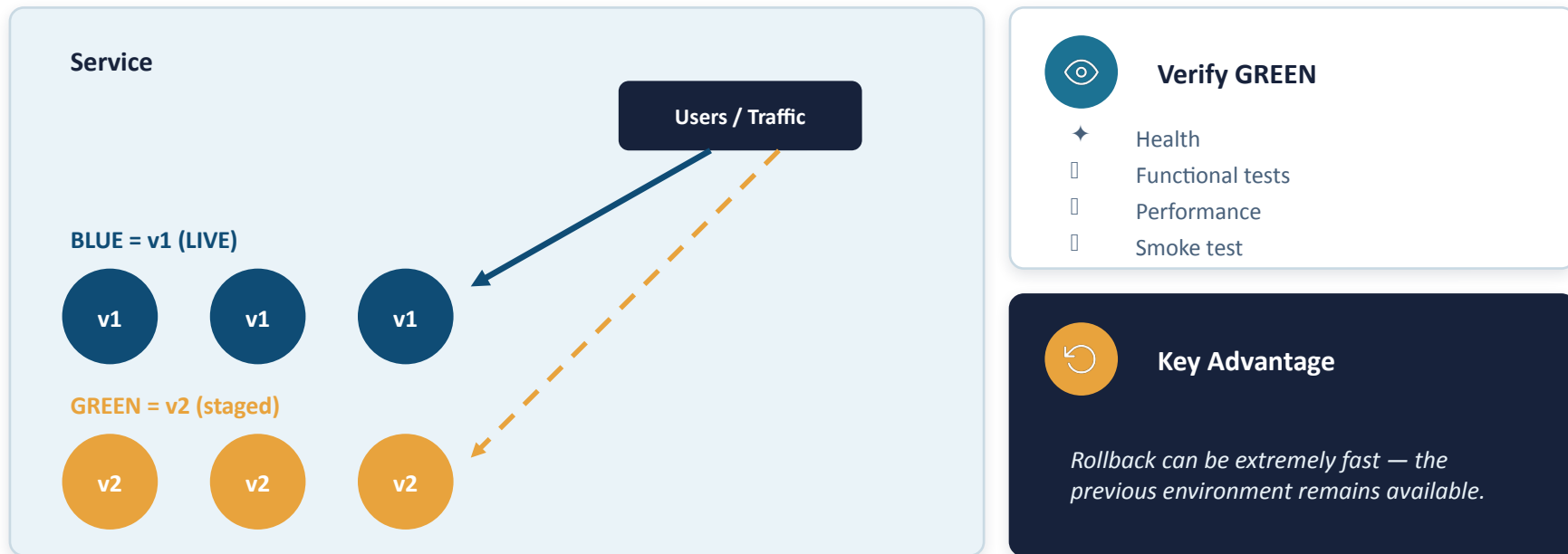


Harness Perspective

Harness deploys the manifest and monitors whether Kubernetes reaches the required steady state.

Blue Green Deployment

Two full production environments — only one receives live traffic at a time.



Harness's Kubernetes Blue Green implementation uses separate pod sets and Kubernetes Services to control which set receives production traffic.

Blue Green: Traffic Switching

Blue Green is not magic — the critical mechanism is the Kubernetes Service selector.

```
selector:  
  app: student-api  
  version: blue
```

BEFORE

Service → version = blue

BLUE

BLUE

BLUE

AFTER SWAP

Service → version = green

GREEN

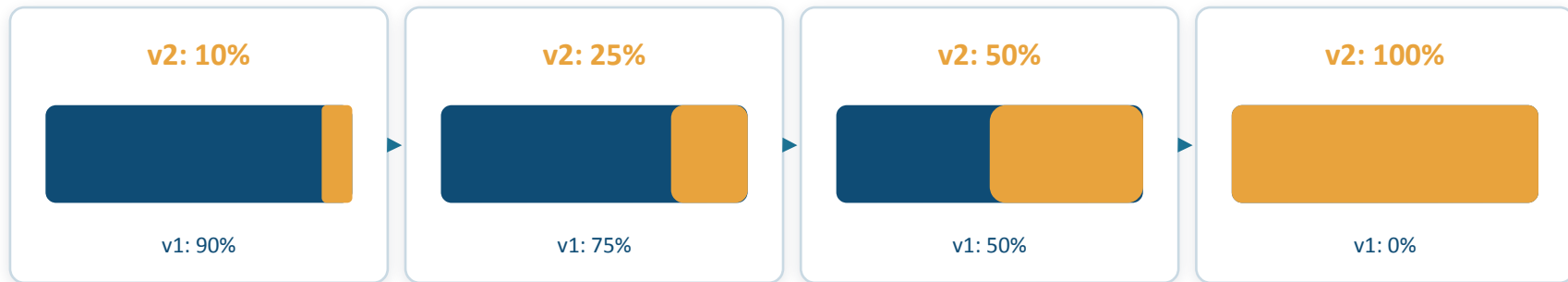
GREEN

GREEN

Teaching point: Blue Green is fundamentally a traffic routing problem — Harness switches traffic between blue and green pod sets using Service selectors.

Canary Deployment

Release to a small audience first, observe, then progressively increase exposure.



↓ Verify at every stage ↓

Monitor at each stage



Error rate



Response time



CPU



Memory



HTTP 5xx



Business metrics

Harness's Kubernetes Canary strategy supports a canary phase followed by a rolling update to complete rollout.

THEY ARE NOT THE SAME

Canary vs Rolling

Rolling	Canary
Gradually replaces pods	Exposes a controlled subset
Mainly infrastructure-oriented	Risk-management oriented
No explicit traffic % required	Often percentage-based
Usually simpler	More verification required
Lower operational complexity	Higher control
Good default strategy	Good for high-risk releases

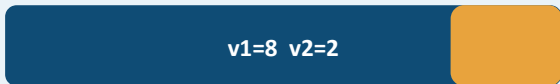
Key concept: Rolling controls replacement of instances. Canary controls exposure to the new version.

Progressive Canary

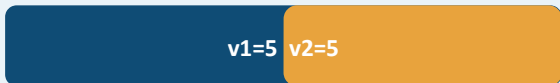
A Kubernetes canary variant that rolls out in percentage phases within a fixed replica budget.

Desired replicas = 10

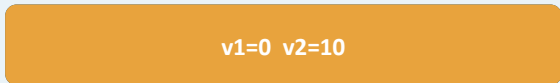
Stage 1



Stage 2



Stage 3



Why This Matters

Traditional canary may temporarily require additional pods.

Progressive canary shifts replicas while keeping the total approximately within the declared replica budget.

Introduced 2026 · may require a feature flag depending on account



Trainer Note: For the lab, keep standard Canary as the primary teaching concept; introduce Progressive Canary as an advanced, current Harness capability.

Deployment Strategy Comparison

Requirement	Rolling	Blue Green	Canary
Simple deployment	✓	~	~
Low additional resources	✓	✗	~
Fast rollback	~	✓	✓
Controlled traffic exposure	✗	✓	✓
Production risk reduction	Medium	High	Very High
Suitable for critical release	✓	✓	✓✓

Standard application

→ Rolling

Need instant traffic switch

→ Blue Green

Need controlled production exposure

→ Canary

DEPLOYING ≠ SUCCESSFULLY RELEASING

Deployment Verification

A deployment can technically succeed and still be bad for users.

